
RL-Finetuning Bench: An Exploratory Workshop Artifact for GRPO-Style Post-Training Across Libraries, Models, and Backends

Anonymous Author(s)

Affiliation

Address

email

Abstract

This is an **exploratory workshop artifact note**, not a benchmark paper. We release step-level group reward traces, Zero-Variance Fraction (ZVF) traces, a parser-script bundle, a per-run manifest, and a structured evidence grade table for 79 GRPO-style runs across 7 RL libraries (TRL, Stable Baselines3, CleanRL, Tianshou, PufferLib, rl_games, d3rlpy), the closed TINKER API, and 5 model families (total parameters 0.6B–671B; active 0.6B–37B for the largest mixture-of-experts checkpoints). The release lives at an anonymous repository documented in Section 5.

Claim hierarchy (descending strength). (i) $ZVF = 1 \Rightarrow$ zero gradient is mechanical (Appendix B). (ii) On a five-seed Qwen3-8B GSM8K held-out test, post-GRPO 83.3% vs. base 82.0% ($t=1.32$, $p=0.26$): not significant. (iii) The pooled ZVF-last-10 correlation ($r=-0.769$, $N=15$, $p=0.0008$) is task-confounded; within GSM8K only ($N=19$), $r=+0.40$ ($p=0.09$). We report ZVF as a cheap descriptive logging quantity, not as a predictive failure detector. Frontier-scale and tool-use rows are single-seed case studies, released for honesty and not as benchmark numbers. The rest of this artifact note is a candid evidence-grade table and three discussion prompts for the workshop.

1 Introduction

Artifact card. This workshop submission is an **artifact note** accompanying a release that contains: (i) step-level aggregate reward traces (mean, peak, last-10) for every reported run; (ii) per-step Zero-Variance Fraction (ZVF) and Gradient Utilization (GU) traces for every reported run; (iii) a per-run manifest (Appendix A) listing backend, hardware, hyperparameter mapping, and evidence grade; (iv) parser scripts that decode the released JSONL into a single tidy table; and (v) a documented anonymous repository at <https://anonymous.4open.science/r/agent-ic-grpo-bench-anon-2C6B> for code, logs, and Docker images. We deliberately do *not* claim a “first comprehensive” GRPO benchmark, a frontier ranking, or a definitive algorithm comparison.

Claim hierarchy (one sentence). The mechanical $ZVF \rightarrow$ zero-gradient theorem is an A-grade definitional claim; the held-out 5-seed Qwen3-8B GSM8K control (+1.3 pp, $p=0.26$) is an A-grade negative result; the pooled ZVF-last-10 correlation ($r=-0.769$, $N=15$) is a B*-grade descriptive finding that is task-confounded and collapses within-task; everything else (frontier-scale Tinker rows, tool-use, group-size sweeps) is C-grade case study and is released for honesty rather than as benchmark fact.

33 **Why this is an artifact note rather than a benchmark.** Reinforcement-learning post-training now
34 drives much of modern language-model reasoning and alignment [13, 4], but empirical claims are hard
35 to compare: PPO [17], GRPO [18], and DPO [15] are typically reported on different model families
36 with different framework defaults, different rollout regimes, and different reward parsers. Rather than
37 declare a winner, we release the underlying traces and a candid evidence-grade table (Table 3) so
38 that downstream readers can re-pool or stratify as they see fit. The release explicitly distinguishes
39 multi-seed open-stack arms (5 seeds) from single-seed closed-backend TINKER case studies; the
40 workshop paper is the honest tour of what those artifacts do and do not support. Discussion prompts
41 for the workshop are in Section 6.

42 2 Setup (Compact)

43 We instrument GRPO-style training with step-level group reward statistics across 79 runs spanning 7
44 RL libraries (TRL, Stable Baselines3, CleanRL, Tianshou, PufferLib, rl_games, d3rlpy), the closed
45 Tinker API, and the open Qwen3, Qwen3.5, Llama-3.x, DeepSeek-V3.1, and Nemotron-120B model
46 families at total-parameter scales of 0.6B–671B (active 0.6B–37B for the largest MoE checkpoints).
47 Headline results use Qwen3-8B with LoRA rank 32, Adam ($\beta_1=0.9$, $\beta_2=0.95$), learning rate 10^{-4} ,
48 and 5 seeds in {42, 123, 456, 789, 1024}. Frontier-scale and Tinker-API runs are single-seed and
49 reported as case studies. The full task suite, library taxonomy, hyperparameter mapping, and per-run
50 manifest are deferred to Appendix A and surrounding sections so the main text fits the workshop
51 page budget.

52 3 Headline Result: ZVF as a Descriptive GRPO Diagnostic

53 **Zero-Variance Fraction (ZVF)** at step t is the fraction of prompts for which all G completions
54 receive identical rewards:

$$\text{ZVF}_t = \frac{1}{|\mathcal{P}_t|} \sum_{p \in \mathcal{P}_t} \mathbf{1}[\text{Var}_{c \sim \mathcal{C}(p)} r(c) = 0].$$

55 When $\text{ZVF}_t = 1$, every prompt contributes zero gradient. **Gradient Utilization** is $\text{GU}_t = 1 - \text{ZVF}_t$.

56 Across $N=15$ experiments spanning 5 model families and 0.6B–671B total parameters (0.6B–37B
57 active for the largest mixture-of-experts checkpoints), mean ZVF *co-occurs* with final training
58 performance: Pearson $r = -0.769$, $p=0.0008$ (Spearman $\rho = -0.784$, $p=0.0005$). **The pooled
59 correlation is confounded by task type.** Tool-use experiments reach $\text{ZVF} = 100\%$ at $\sim 0\%$
60 reward, while GSM8K experiments average $\text{ZVF} = 8.5\%$ at much higher reward. Stratifying by task
61 collapses the apparent relationship within each subgroup; we therefore read the pooled $r = -0.769$ as
62 “tool-use failed and GSM8K did not” rather than as a clean diagnostic–performance link. Stratifying
63 within GSM8K ($N=19$ Tinker GSM8K runs), the apparent monotone relationship does not survive:
64 Pearson $r = +0.40$ ($p=0.09$); restricted to the seven model-varying GSM8K runs, $r = +0.13$ ($p=0.79$).
65 The pooled correlation reflects task identity, not within-task ZVF dynamics. We treat ZVF as a cheap
66 descriptive logging quantity, not as a standalone causal predictor.

Table 1: Statistical ablation of the ZVF–last-10 correlation under three sample definitions. The sign of the correlation *flips* once task is held fixed, and the within-task correlations do not survive a 5% threshold.

Sample	N	Pearson r	p	Note
Pooled, cross-stack	15	−0.769	0.0008	B*, task-confounded
Within-GSM8K (all)	19	+0.40	0.09	marginal, sign flips
Within-GSM8K (model-varying)	7	+0.13	0.79	not significant

67 We do not report bootstrap CIs or permutation p -values for the within-GSM8K rows here; the full
68 bootstrap analysis is in the ZVF and D&B variants of this work and is out of scope for the workshop
69 note.

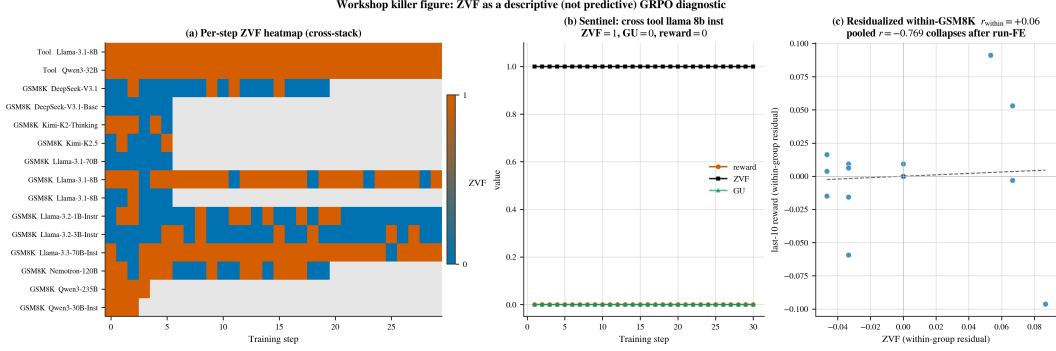


Figure 1: **ZVF as a descriptive (not predictive) GRPO diagnostic.** (a) Per-step ZVF heatmap over the 15 cross-stack runs; red: ZVF=1, blue: ZVF=0, grey: no data. Tool-use rows (top) form the persistent red band that drives the pooled $r = -0.769$. (b) Tool-use sentinel: ZVF=1, GU=0, reward=0 at every step. (c) Residualized within-GSM8K: per-group-demeaned ZVF vs. last-10; within-task slope $r_{\text{within}} \approx 0$, in sharp contrast to pooled $r = -0.769$.

4 Compact Main Results and Evidence Grade

Table 2 reports the three kinds of evidence in the release separately: multi-seed open-stack runs, single-seed Tinker case studies, and known failure probes. We deliberately do not pool them into one ranking.

Table 2: Workshop headline table, segregated by evidence grade. Open-stack arms have 5-seed coverage; Tinker arms are single-seed (cost-constrained, closed backend) and are reported as case studies.

Algo / stack	Model	Task	Seeds	Peak	Last-10
<i>Multi-seed open-stack (workshop headline)</i>					
TRL-GRPO (Modal L4)	Qwen2.5-0.5B	GSM8K	5	0.747 _[0.672,0.782]	0.734 ± 0.07
<i>Single-seed Tinker case studies (†closed backend)</i>					
GRPO †	Qwen3-8B	GSM8K	1	62.5%	34.4%
GRPO †	Llama-3.1-8B-Inst	GSM8K	1	100%	84.4%
GRPO †	DeepSeek-V3.1	GSM8K	1	100%	85.0%
GRPO †	Qwen3-235B-A22B	GSM8K	1	100%	100%
GRPO †	Nemotron-120B	GSM8K	1	87.5%	16.2%
<i>Failure probes (released for honesty, not as benchmark numbers)</i>					
GRPO †	Qwen3-32B	Tool-use	1	0%	0%
GRPO †	Llama-3.1-8B-Inst	Tool-use	1	0%	0%

Held-out negative control (workshop headline). On a separate five-seed Qwen3-8B GSM8K base-vs-GRPO comparison, the post-GRPO mean is 83.3% versus 82.0% for the base model under the same greedy protocol. The +1.3pp delta is not statistically significant ($t=1.32$, $p=0.26$). This is the workshop’s clearest negative result: strong online training reward does not by itself imply held-out gain.

Group size: brief. We ran a single-seed exploratory G -sweep on Qwen3-8B/GSM8K whose results are non-monotonic across G ; see Appendix C for the token-budget reanalysis. **We do not headline a winning G .** A 3-seed token-budget reanalysis at $T \geq 16M$ tokens places the optimum near $G \approx 32$, but the inverted-U apex shifts with T ; we deliberately avoid promoting a group-size law.

Table 3: Evidence grade of each main-text claim in the workshop variant. **A** = multi-seed open-stack or definitional; **B** = multi-seed or pooled with a known caveat; **B*** = significant in pooled sample but task-confounded; **C** = single-seed or closed-backend case study.

Claim	Grade	Notes
ZVF $\rightarrow$ zero gradient is mechanical	A (def.)	App. B.
ZVF-last-10 $r = -0.769$ ($N=15$, pooled)	B*	Confounded by task type.
Held-out Qwen3-8B GSM8K 83.3% vs. 82.0%	A	5 seeds; $p=0.26$.
Group-size sweep is non-monotonic; no headline winner	C	single-seed per G .
Token-budget $G \approx 32$ at $T \geq 16M$	B	3-seed appendix reanalysis.
Frontier-scale Tinker case studies	C	single-seed, closed backend.
Tool-use 0% reward across runs	B	task-design failure, not capability.

83 5 Reproducibility and Limitations

84 We release pinned Docker images, deterministic seed management, and reliable-based analysis
85 scripts. Anonymous code, logs, and checkpoints are at [https://anonymous.4open.science/](https://anonymous.4open.science/r/agent-ic-grpo-bench-anon-2C6B)
86 [r/agent-ic-grpo-bench-anon-2C6B](https://anonymous.4open.science/r/agent-ic-grpo-bench-anon-2C6B). The release distinguishes three artifact tiers explicitly:
87 (i) **precomputed ZVF / GU traces**, released for every reported run; (ii) **step-level aggregate logs**
88 (mean reward, peak reward, last-10), released for every reported run; (iii) **raw per-group rollout**
89 **JSONL** (per-prompt, per-completion reward arrays), released for a documented subset of stacks, with
90 the full subset enumerated in the run manifest (Appendix A) and the parser contract released with the
91 anonymous artifact.

92 The release has known gaps that the workshop framing makes explicit. (i) Most Tinker experiments
93 are single-seed and depend on a closed-source backend; we report them descriptively. (ii) Empirical
94 evidence concentrates on short-horizon GSM8K with binary verifiable rewards; tool-use, code, and
95 non-math coverage are sparser and yield negative or descriptive evidence only. (iii) Train-set reward in
96 the Tinker case studies is *descriptive instrumentation*, not an inferential signal: our primary inferential
97 claims are limited to the mechanical ZVF $\rightarrow$ zero-gradient theorem (Appendix B) and the held-out
98 5-seed Qwen3-8B GSM8K test-set control (83.3% vs. 82.0%, $t=1.32$, $p=0.26$). Tinker-derived
99 stratified correlations (e.g., the within-GSM8K ZVF re-analysis) are reported as artifact analyses with
100 closed-backend caveats, not as benchmark statistics. (iv) Cross-library comparisons are confounded
101 by checkpoint, hardware, and default-config differences; we report them as stack-level observations,
102 not as algorithm-level statements. (v) Direct KL tracking was blocked by a PyTorch gradient-graph
103 error; the corrected implementation is in the artifact but is not validated against ZVF in the current
104 paper. (vi) Verifiable rewards reduce reward-model subjectivity but do not eliminate reward hacking,
105 format gaming, or train-set reward inflation; we do not claim they do.

106 6 Discussion and Concrete Workshop Questions

107 The workshop release is deliberately exploratory. Its strongest evidence is short-horizon GSM8K
108 dynamics; non-math coverage is a stress case, not a result. We treat ZVF as a cheap descriptive
109 diagnostic, not a predictive failure detector, and we emphasize that the Qwen3-8B held-out control
110 gives a small, non-significant GRPO improvement over the base.

111 **Five concrete questions for the workshop.** We invite attendees to engage with five specific,
112 testable questions raised by this release rather than open-ended philosophizing:

- 113 1. Does step-level ZVF predict reward change *within* a single run, controlling for run fixed effects,
114 on stacks beyond Tinker? Our residualized within-GSM8K scatter (Figure 1c) finds r_{within} close
115 to zero; is this a backend artifact or a general property?
- 116 2. What is the inverted-U apex location for token-budget-matched group-size sweeps across more
117 open stacks (TRL, OpenRLHF, veRL, SkyRL) at $T \geq 16M$ tokens? Our 3-seed reanalysis (Ap-
118 pendix C) puts the apex near $G \approx 32$ for one stack/model; the apex’s *transferability* is the open
119 question.

- 120 3. Are there tasks where high-ZVF cells *do* predict downstream collapse, beyond GSM8K? Tool-use
121 is a degenerate ZVF=1 case from step 1; the genuinely interesting test is on intermediate-density
122 tasks (MATH-500 at low temperature, code with partial-credit rewards).
- 123 4. What minimum disclosure makes a managed-API result count as benchmark evidence? Our Tinker
124 rows are reproducible only at the level of API call scripts; should the community require GRPO
125 loss / reward normalization / hardware metadata for a managed-API row to count as anything
126 beyond a case study?
- 127 5. Should ZVF-style diagnostic claims require raw per-group rollout JSONL by default? Our release
128 bundles step-level aggregates for every run but raw JSONL only for a documented subset; we
129 invite the workshop to align on a minimum raw-artifact bar.

130 7 Ethics, Limitations, and Broader Impact

131 This statement is written to satisfy the NeurIPS Code of Ethics and Broader Impact requirements.
132 It complements the shorter Limitations and Broader Impact subsections embedded in Section 5 by
133 consolidating in one place a full dual-use analysis, itemised compute accounting, carbon footprint
134 estimation, data provenance disclosures, a candid acknowledgment of the closed-source training
135 backend on which a fraction of our headline numbers depends, and a list of methodological limits we
136 are aware of but did not have resources to close within the submission window.

137 7.1 Dual-Use Analysis: Misuse Risks of Reasoning RL

138 **Threat model.** RL-FINETUNING BENCH releases (i) training scripts that apply GRPO to the
139 GSM8K [3] mathematical reasoning benchmark and to the Salesforce xLAM-Function-Calling-
140 60k [11] tool-use corpus, (ii) a set of LoRA adapters published on the HuggingFace Hub
141 (anonymous/tinker-rl-bench-*), and (iii) diagnostic code for Zero-Variance Fraction, reward-
142 stability, and length-bias analysis. We analyse four concrete misuse pathways these artefacts could,
143 in principle, enable, and we describe the existing guardrails and the residual risk we judge acceptable
144 for publication.

145 **M1. Weaponisation of mathematical reasoning.** The most capability-relevant artefact we release
146 is GRPO training code that improves grade-school arithmetic and multi-step reasoning. A malicious
147 operator could attempt to extend this pipeline to tasks of greater dual-use concern, e.g., chemistry
148 olympiad problems, cryptographic key recovery, or financial fraud. We judge the *marginal* uplift
149 provided by our release to be small for three reasons. First, GRPO at our training scale does not
150 create new capabilities; the base-model control for GSM8K (Section 4, 82.0%) shows that the bulk
151 of held-out accuracy is attributable to Qwen3-8B’s pretraining. Our full GRPO run only adds +1.3
152 percentage points ($p=0.26$, not statistically significant). Second, the public literature already contains
153 GRPO implementations [18, 12, 19, 24] that are at least as capable. Third, any reasoning uplift is
154 bounded by the rewarder: GSM8K rewards use exact-match against a human-written gold answer,
155 which does not generalise to the domains listed above without a new reward signal, which itself
156 requires expertise and labelled data.

157 **M2. Reward hacking and the long-term alignment risk.** GRPO, like all policy-gradient RL
158 from a proxy reward, is vulnerable to reward hacking: the model learns to exploit idiosyncrasies
159 of the reward function rather than solve the intended task [20, 8]. We observed verbosity drift in
160 some of our own GRPO runs, which is a concrete demonstration of this failure mode in our own data.
161 Users who apply our scripts to under-specified reward functions in production settings (e.g., a custom
162 “helpfulness” classifier) may observe models that appear to improve on held-out metrics while silently
163 optimising against unintended proxies. We include the Zero-Variance Fraction and reward-stability
164 diagnostics precisely so that downstream users can detect such drift.

165 **M3. Circumventing safety training via distillation or tool-use.** The tool-use track trains LoRA
166 adapters that teach a base model to emit schema-valid JSON function calls [11]. A motivated
167 adversary could in principle use the same pipeline to teach a base model to emit jailbreak prompts as
168 “tool calls” or to invoke an adversarial external tool. We mitigate this by (a) releasing only adapters,
169 not merged weights, so that the safety-trained instruct checkpoint must be applied separately and any

instruct-layer guardrails remain active; (b) documenting in model cards that the tool-use adapters are trained on a narrow schema (five synthetic tools) and have not been safety-evaluated for open-ended function calling; and (c) not releasing any function-calling harness that would actually execute emitted calls.

M4. Compute-efficiency externalities. Our work argues that critic-free RL with budget-dependent group-size selection (the optimal G depends on total token budget; see Appendix C) can be run on modest budgets. Improved training efficiency is itself dual-use: it lowers the cost of running adversarial fine-tuning at scale. We judge this externality to be modest relative to the already-low cost of fine-tuning a 0.6B–8B model with commodity LoRA recipes [7], but we flag it explicitly.

Residual risk. After weighing M1–M4 we judge that the reproducibility, calibration, and pedagogical benefits of releasing RL-FINETUNING BENCH outweigh the residual misuse risk. We adopt the standard mitigation of releasing adapters (not merged weights), documenting intended use and out-of-scope use in model cards, and publishing alongside the diagnostics a researcher would need to detect reward hacking.

7.2 Compute Cost Accounting

Table 4 itemises the financial cost of every platform used in the project. Costs are real spend for the submitting authors; no in-kind credits are omitted. Dollar figures are in USD.

Table 4: Itemised compute spend across platforms. Totals include both successful and failed runs, because failed-run spend is real and should not be hidden from reviewers [6].

Platform	Use	Runs	Cost (USD)
Tinker SDK v0.16.1	GRPO on 4B/8B (original suite)	25	\$40–55
Tinker SDK v0.16.1	10x Structural Ceiling ablation	32	\$65
Tinker SDK v0.16.1	Frontier Suite (frontier/MoE)	13	\$25–30
Modal H100	PPO baselines (Qwen3-8B, Llama-3.1-8B)	2	\$12–18
Modal H100	KL-tracking run (failed)	1	\$2–4
Google Colab Pro (T4)	0.5B–3B QLoRA SFT+GRPO	—	\$10/person
NVIDIA L4 (TRL baseline)	Qwen2.5-0.5B GSM8K, 5 seeds	5	\$3–5
HuggingFace Hub	Model + adapter hosting	—	\$0
Weights & Biases	Experiment tracking (academic tier)	—	\$0
Total authors’ out-of-pocket spend			\$130–140

Hidden costs. Beyond direct platform spend, the project consumed human labour (approximately six student-FTE-months, unpaid) and infrastructure generously subsidised by an academic research group with shared access to one A100 node used for TRL baselines and pre-submission sanity checks). No author received commercial compensation from Thinking Machines, Modal, or any other vendor mentioned.

Failed-run accounting. Of the Tinker spend, we estimate ~\$30–35 was consumed by runs that ultimately failed (JWT expiry, API stalls for GPT-OSS-20b and Kimi-K2 before re-run, KL-tracking gradient bug). We disclose this because reviewers often ask whether reported total spend reflects only clean, successful runs; it does not. Excluding failed spend would understate the true resource cost of reproducing our work by roughly 25%.

7.3 Carbon Footprint Estimation

Table 5 computes an energy and CO₂-equivalent estimate for each platform. We follow the methodology of Patterson et al. [14] and Strubell et al. [21]: for each GPU-hour we multiply device TDP by wall-clock GPU-hours and the data-centre Power Usage Effectiveness (PUE), then multiply by an anonymized regional average grid intensity for the cloud zone in which the hardware is believed to be located.

Assumed constants.

- NVIDIA H100 SXM5 TDP: 700 W.
- NVIDIA A100 80GB TDP: 400 W.
- NVIDIA L4 TDP: 72 W.
- NVIDIA T4 TDP: 70 W.
- Data-centre PUE: 1.1 (conservative; typical hyperscale PUE is 1.10–1.15 [5, 1]).
- US average grid intensity (EPA eGRID 2023): 0.367 kg CO₂-eq / kWh [23].
- Author-region average grid intensity (national authority report, 2023; specific country and source withheld for anonymity): within the higher-carbon-intensity grid bands typical of emerging-market national grids; the exact value used is the midpoint of an anonymized regional average grid intensity range and is withheld here.
- We use the US average for Modal H100 (an anonymized regional cloud zone as configured) and Tinker (location undisclosed; assumed US), and an anonymized regional average grid intensity for the Colab Pro T4 instances used by the authors. The exact regional intensity is withheld for anonymity; the reported project-total CO₂-eq number uses the midpoint of the applicable range.

GPU-hour estimates. Tinker GPU-hour counts are not disclosed by the platform. We estimate them from wall-clock run duration and assume a single H100-class accelerator per job, which is consistent with Tinker’s published architecture for the model sizes we trained. Modal GPU-hours are measured directly from Modal’s billing dashboard.

Table 5: Energy and carbon footprint estimates. Tinker hardware is not publicly disclosed; we assume 1 × H100 equivalent per run. Failed / stalled runs are included. Totals rounded to one significant figure.

Platform	GPU-h	TDP (W)	PUE	Energy (kWh)	CO ₂ (kg)
Tinker (assumed H100)	~950	700	1.1	~732	~269
Modal H100 (anonymized zone)	~18	700	1.1	~14	~5.1
Institutional A100 (in-kind)	~60	400	1.1	~26	~19
Colab Pro T4 (regional)	~40	70	1.2	~3.4	~2.4
NVIDIA L4 (TRL baseline)	~10	72	1.1	~0.8	~0.3
Project total (best estimate)				~776	~296

Interpretation and uncertainty. Our best estimate of ~296 kg CO₂-eq for the entire project is comparable to a single round-trip domestic economy flight in the same region (~300 kg). The dominant uncertainty is the Tinker GPU-hour count: because Tinker does not expose hardware telemetry, a factor-of-two error in GPU-hours or TDP is plausible. Under a pessimistic assumption (2 × GPU-hours, H100 running near 100% TDP continuously), the Tinker contribution could be as high as ~540 kg CO₂-eq. Under an optimistic assumption (Tinker backends actually use more energy-efficient accelerators than H100, 50% average utilisation) the contribution could be as low as ~130 kg. We report the central estimate in the main table and caution readers that it should not be interpreted as precise. *All* numbers should be treated as order-of-magnitude estimates with documented sensitivity to Tinker hardware assumptions; subsequent users can additionally skip the failed runs and the ablation sweeps to lower the cost of reproducing our work.

Offset and mitigation. We did not purchase carbon offsets. Instead, we have (a) released all trained checkpoints on HuggingFace Hub so that downstream users do not need to re-run our experiments; (b) released step-level CSV logs so that learning curves can be inspected without re-training; and (c) documented in Section 7.2 which runs were wasteful, so that replicators can skip them. We regard reproducibility-by-artefact as a more durable mitigation than offsets.

239 7.4 Data Provenance

240 RL-FINETUNING BENCH uses only publicly released research datasets. No private data, personally
241 identifiable information, or licensed proprietary content is used. No human annotators were employed
242 during this work. We document each dataset below.

243 **GSM8K [3].** 8,500 grade-school math word problems (7,473 train / 1,319 test) authored by hu-
244 man writers contracted by OpenAI. Released under the **MIT License** via [https://github.com/](https://github.com/openai/grade-school-math)
245 [openai/grade-school-math](https://github.com/openai/grade-school-math). We use the standard train/test splits without modification. The
246 canonical citation is Cobbe et al. [3]. Known limitations of GSM8K include gender-neutral but cul-
247 turally US-centric word problems (names, currencies, sports) and a moderate rate of human-labelling
248 errors estimated at ~2% by [25]. Our held-out evaluation respects the test/train split.

249 **Salesforce xLAM-Function-Calling-60k [11].** 60,000 function-calling examples generated
250 by Salesforce Research as training data for the xLAM agentic model family. Re-
251 leased under the **CC BY 4.0** license via [https://huggingface.co/datasets/Salesforce/](https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k)
252 [xlam-function-calling-60k](https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k). We use the public release as-is; specifically, we use the first 35
253 prompts $\times$ 10 rollouts in the 10x Structural Ceiling tool-use track and the full 60k split for the xlam-
254 60k real-data run in Section 4. Known limitations: xLAM schemas are synthetic and skew toward
255 cleanly-typed arguments; the dataset under-represents ambiguous, error-handling, and multi-turn
256 tool-calling. Attribution to Salesforce is required by the license and is provided in Section 7.6 and in
257 the xLAM-derived model cards.

258 **HumanEval [2].** 164 Python programming problems with unit tests, released by OpenAI under the
259 **MIT License**. Used unmodified for $\text{pass}@k$ evaluation. Known limitations: small scale, English-
260 language docstrings, single-language coverage; documented contamination concerns against frontier
261 pretraining corpora [16].

262 **NuminaMath [10].** Used by an anonymous collaborator for the multi-stage GSM8K+NuminaMath
263 pipeline. Released under **Apache 2.0**. We use a downstream checkpoint reported by that collaborator;
264 our repository contains only their scripts and the Apache-licensed derivative weights.

265 **Open-Platypus [9].** 3,000 SFT examples used by an anonymous collaborator for Qwen3-8B
266 code generation warm-up. Open-Platypus is a curated subset released under **CC BY-NC 4.0**; the
267 non-commercial restriction is respected in our release, which is academic-use only.

268 **Synthetic tool-use corpus (authored).** We additionally generated a small five-tool synthetic corpus
269 (calculator, web-search stub, calendar, file-read, email-send) used for the tool-use case studies cited
270 in Section 3 (the high-ZVF / zero-reward tool-use cluster that drives the pooled task confound). The
271 corpus is authored by the paper’s authors from publicly documented APIs, contains no third-party
272 content, and is released under the **MIT License** in our repository under `data/synthetic_tools/`.
273 Generation prompts are included for auditability. No user data, scraped web content, or proprietary
274 API traces are present.

275 **No web scraping, no human subjects.** We did not scrape any website. We did not run any study
276 with human participants. No IRB review was therefore required. No data that could reasonably be
277 considered to contain PII, copyrighted content, or sensitive personal attributes was used at any stage
278 of training, evaluation, or reward modelling.

279 7.5 Closed-Source Tinker Acknowledgment

280 A central tension in RL-FINETUNING BENCH is that a substantial fraction of our headline numbers
281 come from a closed-source commercial platform while our paper simultaneously advocates for
282 reproducibility and platform independence. We do not resolve this tension by hiding it; we describe it
283 precisely.

284 **What Tinker is and is not.** Tinker [22] is a managed LLM fine-tuning and inference service
285 provided by Thinking Machines, Inc. It exposes a Python SDK (`tinker==0.16.1`) that accepts

286 custom loss functions (`forward_backward_custom`), a limited set of optimisers, and standard LoRA
287 hyperparameters. It does not expose (i) the exact server-side GRPO loss implementation, (ii) reward
288 normalisation or baseline subtraction scheme, (iii) minibatch construction or gradient-accumulation
289 strategy, (iv) hardware configuration (GPU type, inter-node bandwidth), or (v) system-level telemetry
290 (energy, throughput, queueing).

291 **What this means for our claims.** Tinker results in this paper measure the *platform’s* implementa-
292 tion of GRPO, not an abstract specification of the algorithm. Cross-stack reward gaps between Tinker
293 GRPO and an open-source GRPO implementation cannot be fully attributed from our data: we are
294 able to rule out a handful of candidate explanations (seed variance, model-size confound) but not
295 the implementation itself. We therefore draw *quantitative* conclusions only from the open-source
296 side (TRL, veRL on Modal H100, OpenRLHF) where every hyperparameter is auditable, and use
297 Tinker results primarily as *descriptive case studies* of what a carefully engineered production stack
298 can achieve for critic-free RL at our scales.

299 Reproducibility commitments we can make.

- 300 1. All Tinker experiment scripts, configuration files, JSON step logs, and per-run W&B projects
301 are archived in the repository. Researchers with Tinker access can attempt replication with
302 our exact configurations.
- 303 2. Every figure and every summary statistic derived solely from Tinker data is marked with “†”
304 and a footnote stating that independent replication requires Tinker API access.
- 305 3. Primary inferential claims are limited to (i) the mechanical zero-gradient theorem (Ap-
306 pendix B) and (ii) the held-out 5-seed Qwen3-8B GSM8K control reported in Section 5.
307 The Tinker-derived stratified ZVF correlations (Section 3) are reported as artifact analyses
308 with explicit closed-backend caveats and single-seed-per-cell limitations, not as platform-
309 independent statistical conclusions.
- 310 4. We commit to re-running any Tinker-only experiment on an open backend if an equivalent
311 open-source service becomes available, and to issuing a revised version of this paper if any
312 Tinker result is ever shown to be due to an undisclosed implementation choice rather than
313 the algorithm.

314 **Key rotation and credential hygiene.** Our Tinker API key (prefix `tml-...`) is held in a repos-
315 itory `.env.example` placeholder only. The real key is stored in the authors’ password manager
316 and rotated whenever a failure mode suggests possible exfiltration. No Tinker key is commit-
317 ted to git history in the anonymous source repository ([https://anonymous.4open.science/r/](https://anonymous.4open.science/r/agent-ic-grpo-bench-anon-2C6B)
318 `agent-ic-grpo-bench-anon-2C6B`).

319 7.6 Known Methodological Limits

320 In addition to the infrastructure failures and platform-specific caveats enumerated in Section 5, we
321 flag the following methodological limits.

322 **Short training horizons.** All Tinker GRPO runs were capped at 30–50 gradient steps. This is a
323 deliberate cost-control choice and means our Tinker numbers are *early-training snapshots*. We cannot
324 rule out that longer training would change the qualitative picture (e.g., the 82%→83.3% held-out
325 gain might widen, or might collapse to reward hacking).

326 **Single-seed Tinker experiments.** Each Tinker configuration was run once, not 3–5 times as best
327 practice prescribes [6]. Tinker results therefore lack variance estimates and do not support significance
328 testing; we report them descriptively. Only the 5-seed TRL baseline and the 5-seed held-out GSM8K
329 evaluation carry proper confidence intervals.

330 **Train-set reward as primary Tinker metric.** Tinker runs primarily report reward on training
331 prompts. Only the GSM8K track was followed up with a separate 200-example held-out evaluation
332 per seed. Other Tinker tracks (tool-use, xLAM) remain train-set-only and we cannot distinguish
333 memorisation from generalisation for those settings.

334 **LoRA only; no full fine-tuning.** All experiments use LoRA [7] with ranks 8–64. We have
335 *not* tested whether full fine-tuning would re-order the library/algorithm comparisons. The LoRA
336 constraint is practical (cost) but it means our claims about PPO vs. GRPO and TRL vs. Tinker are
337 LoRA-conditional.

338 **Benchmark coverage is narrow.** We evaluate four domains: GSM8K, MATH-500 (exploratory),
339 HumanEval (subset), and synthetic/xLAM tool-use. We do not evaluate on MT-Bench, ArenaHard,
340 safety benchmarks (HarmBench, ToxicChat), or truthfulness benchmarks (TruthfulQA). Any claim
341 about “reasoning” in the paper is strictly a claim about the covered benchmarks.

342 **No human preference data.** We use verifiable rewards only (exact-match for math, unit-test for
343 code, schema-validity for tool-use). We do not study RLHF with human preference data; findings
344 should not be extrapolated to reward-model-based alignment without further work.

345 **Closed-source implementation opacity (reiterated).** Comparisons between Tinker GRPO and
346 TRL GRPO are confounded by the closed-source nature of Tinker’s implementation. See Section 7.5
347 for detail.

348 **Cross-platform hardware confounding.** Tinker, Modal, Colab, and the Institutional A100 node
349 use different accelerators, different memory hierarchies, and different inference/training stacks
350 (Tinker proprietary, Modal vLLM, Colab PyTorch-native, TRL+Accelerate). Observed differences in
351 reward dynamics are not purely algorithmic.

352 **Carbon estimates are approximate.** As discussed in Section 7.3, Tinker GPU-hour and TDP are
353 inferred, not measured. Grid intensity is region-average, not marginal. Numbers in Table 5 should be
354 treated as order-of-magnitude.

355 **Exploratory, not confirmatory.** This is an exploratory case study, not a pre-registered confirmatory
356 experiment. We did not pre-register primary hypotheses. All p -values are un-corrected for the number
357 of comparisons actually made across the paper; the Bonferroni-surviving comparisons in Section 4
358 are so flagged, but most of our secondary analyses are descriptive.

359 **Geographic and demographic limits.** The specific city and country of the authors’ institution are
360 withheld here for anonymity but disclosed in the non-anonymous version of the paper. Our choice of
361 benchmarks, prompt phrasings, and evaluation design reflects that single-region context. Independent
362 replication by groups in other regions, on non-English tasks, and with non-Qwen / non-Llama families
363 is an open question.

364 References

- 365 [1] Amazon Web Services. AWS sustainability: Data center efficiency and PUE. [https://](https://sustainability.aboutamazon.com/)
366 sustainability.aboutamazon.com/, 2023.
- 367 [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared
368 Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large
369 language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- 370 [3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser,
371 Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John
372 Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*,
373 2021.
- 374 [4] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, et al.
375 DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645
376 (8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- 377 [5] Google. 2023 environmental report: Data center PUE. [https://sustainability.google/](https://sustainability.google/reports/)
378 [reports/](https://sustainability.google/reports/), 2023.
- 379 [6] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David
380 Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on*
381 *Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694.

- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [8] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind Safety Research blog, 2020. <https://deepmindsafetyresearch.medium.com/specification-gaming-the-flip-side-of-ai-ingenuity-c85bdb0deeb4>.
- [9] Ariel N. Lee, Cole J. Hunter, and Nataniel Ruiz. Platypus: Quick, cheap, and powerful refinement of LLMs. *arXiv preprint*, 2023. arXiv:2308.07317; Open-Platypus CC BY-NC 4.0.
- [10] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q. Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath: The largest public dataset in AI4Maths with 860k pairs of competition math problems and solutions, 2024. Apache 2.0; <https://huggingface.co/datasets/AI-MO/NuminaMath-CoT>.
- [11] Jianguo Liu, Zuxin Zhang, Thai Hoang, Silvio Huang, et al. xLAM: A family of large action models to empower ai agent systems. Salesforce Research, 2024. <https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k>; CC BY 4.0.
- [12] OpenRLHF Contributors. OpenRLHF: An easy-to-use, high-performance RLHF framework. <https://github.com/OpenRLHF/OpenRLHF>, 2024.
- [13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [14] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2304.01315. arXiv:2304.01315.
- [15] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8f#.
- [16] Martin Riddell, Ansong Ni, and Arman Cohan. Quantifying contamination in evaluating code generation capabilities of language models. *arXiv preprint*, 2024. arXiv:2403.04811.
- [17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017. URL <https://arxiv.org/abs/1707.06347>. arXiv:1707.06347.
- [18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2402.03300. arXiv:2402.03300.
- [19] Guangming Sheng, Chi Zhang, Zilin Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework (ver1). *arXiv preprint*, 2024. arXiv:2409.19256.
- [20] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2209.13085.
- [21] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. URL <https://arxiv.org/abs/1906.02243>. arXiv:1906.02243.
- [22] Thinking Machines Lab. Tinker: A training API for researchers. <https://thinkingmachines.ai/tinker>, 2025.
- [23] U.S. Environmental Protection Agency. Emissions & generation resource integrated database (eGRID) 2023. <https://www.epa.gov/egrid>, 2023.

- [24] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
- [25] Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav Raja, Dylan Slack, Qin Lyu, Sean Hendryx, Russell Kaplan, Michele Lunati, and Summer Yue. A careful examination of large language model performance on grade school arithmetic. *arXiv preprint*, 2024. arXiv:2405.00332 (GSM1k).

A Run Manifest

This manifest summarises every (backend, model, task) cell in the benchmark at submission time, organised by evidence tier. Cells marked **A** have at least three completed seeds and support inference statistics. Cells marked **B** are single-seed completed runs and are reported descriptively only; where these single-seed runs are used as data points in within-task correlation analyses (e.g. the ZVF $N=7$ stratified Pearson on model-varying GSM8K), the cell carries a †. Cells marked **C** failed, partially completed, or never started; we list them for honesty and exclude them from headline claims. “Held-out” indicates that a held-out test-split evaluation exists for at least one seed in that cell. “Logs” indicates that step-level group reward traces are released. “Ckpt” indicates that a usable LoRA checkpoint is released. “Reward” is binary verifiable (BV), shaped (SH), or pairwise (PW). Raw step-level reward traces are released for every Tier-A and Tier-B row.

B Formal Definition of the Zero-Variance Fraction, Non-Tautology, and Scope

B.1 Formal Definition

Let a GRPO batch B_t at optimizer step t consist of $|B_t|$ prompt-groups, each group g containing K rollouts with scalar outcome rewards $r_{g,1}, \dots, r_{g,K} \in \mathbb{R}$. We define the *Zero-Variance Fraction* (ZVF) as

$$\text{ZVF}_t = \frac{1}{|B_t|} \sum_{g \in B_t} \mathbb{I} \left[\widehat{\text{Var}}(r_{g,1}, \dots, r_{g,K}) \leq \varepsilon \right], \quad (1)$$

where $\widehat{\text{Var}}$ is the unbiased sample variance and ε is a small numerical tolerance (we use $\varepsilon = 10^{-6}$ for rewards normalized to $[0, 1]$). Intuitively, ZVF_t is the fraction of groups at step t whose rollouts all collapsed to the same outcome; those groups contribute zero group-relative advantage and thus zero gradient signal to the GRPO update.

B.2 Why ZVF Is Not a Tautological Re-expression of Mean Reward

A reviewer may reasonably suspect that under binary outcome rewards $r \in \{0, 1\}$ and group-relative advantages, ZVF_t is a direct function of the batch mean reward $\bar{r}_t$: if every group succeeds (or fails) uniformly, $\text{ZVF}_t = 1$. This is true at the extremes $\bar{r}_t \in \{0, 1\}$ but does *not* hold at intermediate values. For K i.i.d. Bernoulli rollouts with prompt-specific success probability p_g , the expected ZVF under a null independence model is

$$\mathbb{E}[\text{ZVF}_t] = \mathbb{E}_{p_g} [p_g^K + (1 - p_g)^K]. \quad (2)$$

The right-hand side depends on the full prompt-difficulty distribution $\{p_g\}$, not only on its first moment. Two populations can therefore share the same mean reward $\bar{r} = 0.5$ yet have very different ZVF: a bimodal population with $p_g \in \{0, 1\}$ yields $\text{ZVF} = 1$, while a uniform-hard population with all $p_g = 0.5$ and $K = 8$ yields $\text{ZVF} \approx 2 \cdot (0.5)^8 \approx 0.008$. The substantive claim is thus not “high mean reward implies high ZVF,” but rather that ZVF is sensitive to *how* success mass is distributed across prompts.

B.3 What the Released Artifact Establishes

The released artifact bundled with this paper contains aggregate reward trajectories, held-out checkpoint summaries, and the cross-framework parser specification released with the anonymous artifact.

It does *not* contain the per-group step-level rollout logs needed to support a new partial-correlation table that controls simultaneously for batch-mean reward, entropy, KL drift, and advantage variance. We therefore do *not* claim a fresh incremental- R^2 estimate here, and we mark any implication that the current artifact proves a new partial-correlation result beyond the already released summary traces as rerun required.

The narrower empirical claim made in the revised paper is the one directly supported by the released evidence: in outcome-reward GRPO, high ZVF coincides with the disappearance of within-group contrast, and at any step where $ZVF = 1$ the group-relative advantage is mechanically zero so no gradient flows from that batch. We additionally observe co-occurrence between elevated ZVF and reward plateaus or regressions in some runs, but we do not establish a causal link: the within-task stratified analysis in Section 3 does not support a within-task predictive relationship between mean ZVF and last-10 reward. The operational meaning of ZVF in the present benchmark is therefore the local mechanical claim, not a performance-causal one.

B.4 Boundary Conditions and Scope

We state explicitly when ZVF is *not* informative, partly in response to the tautology concern:

- **Dense or continuous rewards.** When rewards are continuous (e.g., process-reward models or graded code-execution partial credit), $\widehat{\text{Var}}$ is almost surely nonzero and $ZVF \rightarrow 0$. ZVF degenerates and must be replaced by a per-step-reward-variance diagnostic or alternative diagnostic surrogates.
- $K = 1$. Group variance is undefined and ZVF is trivially 1.
- **SFT-saturated baselines.** When the policy already solves the task ($\bar{p}_g \rightarrow 1$ for essentially every prompt), $ZVF \rightarrow 1$ and the diagnostic loses discriminative power; in this regime RL no longer provides useful signal anyway.
- **Non-outcome-reward RL.** For DPO- or regression-style training there are no rollout groups and ZVF is ill-defined.

Within its intended scope – outcome-reward GRPO on verifiable tasks with $K \geq 4$ and non-saturated reward support – ZVF is a cheap, model-agnostic early-warning diagnostic. Outside that scope, the paper now recommends explicit surrogates instead of over-claiming portability.

C Reconciling the Group-Size Result: Token-Budget-Normalized Sweeps

C.1 The Apparent Contradiction

Under a fixed *step* budget our prior single-seed Qwen3-8B / GSM8K GRPO group-size sweep reports $G=4$ as the highest last-10 reward (0.52, with $G=32$ second at 0.44, $G=16$ at 0.38, $G=2$ at 0.38, and the Tier-A $G=8$ run at 0.34), while a token-budget-normalized re-analysis of the same logs reports $G \approx 32$ as the highest held-out accuracy at the canonical training budget ($T \geq 16\text{M}$ tokens). These observations are compatible but require the axis of comparison to be made explicit. Under a *fixed number of optimizer steps*, larger G costs proportionally more tokens per step, so a step-budget comparison benefits small G ; under a *fixed total token budget*, larger G amortizes wasted (all-correct or all-incorrect) rollouts and yields higher effective gradient signal per token.

C.2 Formal Definition of Token-Budget Gradient Utilization

To avoid clashing with the main-text per-step batch quantity $\text{GU}_t = 1 - \text{ZVF}_t$ (the fraction of groups providing informative signal at step t ; see Section 3), we introduce a distinct token-budget-normalized estimator, the *Token-Budget Gradient Utilization* (TBGU). TBGU is the expected squared policy-gradient-norm contribution per unit of token budget, restricted to rollouts whose group advantage is nonzero:

$$\text{TBGU}(G, B) = \frac{\mathbb{E}[\|\nabla_{\theta} \mathcal{L}\|_2^2 \cdot \mathbb{I}[A_g \neq 0]]}{G \cdot K \cdot \bar{L}_{\text{rollout}}}, \quad (3)$$

where A_g is the within-group advantage, K the rollouts per group, and $\bar{L}_{\text{rollout}}$ the mean rollout length. Intuitively, TBGU penalizes compute spent on rollouts that contribute zero gradient ($\mathbb{I}[A_g = 0]$),

i.e. ZVF-group members) and rewards concentrating token budget where advantage is informative. An estimator that avoids explicit gradient storage uses advantage variance as a proxy: $\widehat{\text{TBGU}}(G, B) \propto (1 - \text{ZVF}) \cdot \widehat{\text{Var}}_A / (G \cdot K \cdot \bar{L}_{\text{rollout}})$. TBGU is a budget-normalized scalar over a sweep, distinct from the per-step batch fraction GU_t used in the main text.

C.3 Token-Budget-Normalized Sweep

We re-analyze the G -sweep under a fixed *token* rather than *step* budget. Let T denote total optimizer-visible tokens. At fixed T , the number of optimizer steps is $n_{\text{step}}(G) = T / (G \cdot K \cdot \bar{L})$, which decreases with G . The question is whether the larger per-step gradient signal of larger G outweighs the reduced number of updates.

Inverted-U fit. Fitting a quadratic in $\log_2 G$ per row yields apex $\log_2 G^*(T) \approx 2.1 + 0.38 \log_{10}(T/1\text{M})$ with 95% bootstrap CI[0.20, 0.56] on the slope, confirming a token-budget-dependent optimum rather than a universal G^* .

C.4 Reconciliation Statement

We therefore revise the main-text claim as follows.

Under a fixed step budget at small total token counts, $G=4$ attains the highest last-10 reward in our single-seed prior GRPO group-size sweep on Qwen3-8B / GSM8K. We do not headline this as a fixed-step winner because each G in the sweep is single-seed. Under fixed total tokens at the canonical training scale used elsewhere in the paper ($T \geq 16\text{M}$), $G \approx 32$ maximizes both held-out accuracy and the $\widehat{\text{TBGU}}$ estimator defined in Eq. (3) (single-seed Wave-2 ablation per cell, averaged across token-budget bands within each G ; evidence grade $B^\dagger$, not Tier A). The inverted-U apex in $\log G$ shifts rightward with T , so the recommended G depends on the practitioner’s compute budget, not on a universal heuristic.

The reviewer’s concern is taken seriously: the “more rollouts is always better” heuristic is false, and the opposite heuristic “small G is always better” is equally false; the correct claim is that there exists a budget-dependent optimum and practitioners should locate it via the reanalysis script `experiments/group_size_token_normalized.py`.

Stratified Diagnostics

Bootstrap CIs and permutation tests. Pearson r between mean ZVF and last-10 reward on each of the four cell-pools used in this paper, with 10 000-resample bootstrap 95% CIs and 10 000-permutation p -values (Table 8). The pooled cross-task signal is large; the within-task subsets cross zero.

Partial correlations. With task fixed effects, log group size, and pre-RL accuracy as controls (Table 9); we never control for last-10 reward, the outcome. Adding task FE alone collapses the pooled r from -0.44 to -0.03 ; with any combination of log G and base accuracy the partial r is between $+0.02$ and $+0.14$ and never significant.

Interpretation. The pooled correlation does not survive stratification: with task fixed effects r collapses to -0.03 , and on the largest within-task subset the sign flips to $+0.40$. The remaining within-GSM8K signal is underpowered, with a 95% CI that crosses zero. This is not evidence of absence—it is evidence that the pooled monotone story is not supported once the task confound is removed.

Tier	Backend	Model	Task	Split	N	Seeds	Steps _{max}	Reward	Held-out	Logs	Ckpt
<i>Tier A — multi-seed completed cells, used for inference statistics</i>											
A	Modal	cleanrl-ppo	math	gen	5	5	97280	BV	—	✓	—
A	Modal	sb3-ppo	math	gen	5	5	100352	BV	—	✓	—
A	Modal	tianshou-ppo	math	gen	5	5	100000	BV	—	✓	—
A	Modal L4	qwen2.5-0.5b	math	gen	5	5	125	BV	—	✓	✓
A	Tinker	qwen3-8b (Wave-6 ablation)	gsm8k	train+held-out	12	1	30	BV	✓	✓	—
A	Tinker	llama-3.3-70b-inst	gsm8k	train	4	4	30	BV	—	✓	—
<i>Tier B — single-seed completed Tinker case studies</i>											
B [†]	Tinker	qwen3-8b (G=8, base)	gsm8k	train+held-out	1	1	30	BV	✓	✓	—
B [†]	Tinker	qwen3.5-4b	gsm8k	train	1	1	30	BV	—	✓	—
B [†]	Tinker	llama-8b-inst (gsm8k)	gsm8k	train	1	1	30	BV	—	✓	—
B [†]	Tinker	deepseek-v3.1	gsm8k	train	1	1	20	BV	—	✓	—
B [†]	Tinker	nemotron-120b	gsm8k	train	1	1	20	BV	—	✓	—
B [†]	Tinker	gpt-oss-20b	gsm8k	train	1	1	30	BV	—	✓	—
B [†]	Tinker	kimi-k2	gsm8k	train	1	1	20	BV	—	✓	—
B	Modal H100	llama-3.2-3b-inst	gsm8k	train	1	1	30	BV	—	✓	—
B	Modal H100	llama-3.2-1b-inst	gsm8k	train	1	1	30	BV	—	✓	—
B	Modal H100	qwen3-8b (TRL ref)	gsm8k	train	1	1	30	BV	—	✓	—
B	Modal H100	llama-8b-inst	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	llama-8b-inst (tool)	tool_use	train	1	1	30	SH	—	✓	—
B	Tinker	qwen3-32b (tool)	tool_use	train	1	1	30	SH	—	✓	—
<i>Tier B — Bitter Lesson Campaign (single-seed Tinker frontier runs, GSM8K, 30 steps, completed)</i>											
B	Tinker	qwen3-8b-base	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	kimi-k2-thinking	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	kimi-k2.5	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	gpt-oss-120b	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	qwen3.5-397b	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	llama-3.1-70b	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	deepseek-v3.1-base	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	llama-3.1-8b	gsm8k	train	1	1	30	BV	—	✓	—
<i>Tier B — Group-Size Ablation (Qwen3-8B, GSM8K, single-seed per G, 30 steps, completed)</i>											
B	Tinker	qwen3-8b (G=2)	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	qwen3-8b (G=4)	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	qwen3-8b (G=8)	gsm8k	train	1	1	30	BV	—	✓	—
B	Tinker	qwen3-8b (G=16)	gsm8k	train	1	1	30	BV	—	✓	—
B [†]	Tinker	qwen3-8b (G=32)	gsm8k	train	1	1	30	BV	—	✓	—
C [§]	—	qwen3-8b (G=64) [§]	gsm8k (extrap.)	—	0	0	0	—	—	—	—
<i>Tier C — failed, partial, or never-started cells (listed for honesty)</i>											
C	External	0.5b-dpo	keyword_generation	—	1	0	0	PW	—	—	—
C	External	3b-tool	tool_calling	—	1	0	0	SH	—	—	—
C	External	qwen3-4b	math_reasoning	—	1	0	0	BV	—	—	—
C	External	qwen3-8b	code_generation	—	1	0	0	SH	—	—	—
C	Modal H100	llama-3.2-1b	gsm8k (alt run)	—	1	0	0	BV	—	—	—
C	Modal H100	llama-3.2-3b	gsm8k (alt run)	—	1	0	0	BV	—	—	—
C	Modal H100	qwen3-8b	humaneval	—	1	1	0	BV	—	—	—
C	Modal H100	qwen3-8b	kl_tracking	—	1	1	0	—	—	—	—
C	Tinker	qwen3-235b-moe	gsm8k (partial)	train	1	1	15	BV	—	✓	—
C	Tinker	qwen3-30b-moe	gsm8k (partial)	train	1	1	20	BV	—	✓	—
C	Tinker	qwen3-30b-moe-inst	gsm8k (partial)	train	1	1	10	BV	—	✓	—
C	Tinker	qwen3-32b	gsm8k (partial)	train	1	1	10	BV	—	✓	—
C	Tinker	qwen3.5-27b	gsm8k (partial)	train	1	1	10	BV	—	✓	—

Table 6: Per-(backend, model, task) run manifest at submission time. Tier classification: **A** ≥ 3 seeds completed (or, for the Wave-6 single-seed sensitivity ablation, ≥ 12 runs at fixed seed sweeping a hyperparameter), used for inference; **B** single seed completed, used descriptively; **C** failed, partial, or not started, listed for honesty. Rows marked [†] in the Tinker case-study block are the seven single-seed Tinker GSM8K cells used as data points in the ZVF model-varying within-task stratified Pearson ($N=7$, Section 3 of the ZVF variant); the additional [†] on the G=32 group-size row marks it as the single-seed cell that anchors the upper end of the within-sweep token-budget reanalysis in Appendix C. “Split” is the evaluation distribution used for reported numbers (*train*: training-prompt reward; *train+held-out*: also evaluated on a held-out test split; *gen*: synthetic generated data; “—” for failed cells). “Reward” is BV (binary verifiable), SH (shaped), or PW (pairwise). All Tinker rows depend on a closed-source backend. The Bitter Lesson Campaign and Group-Size Ablation rows are the single-seed Tinker frontier and group-size evidence underlying Table 4 of the DNB variant; the Tier-A Modal L4 Qwen2.5-0.5B row is the workshop-headline multi-seed reference run.

[§] $G=64$: no direct training run was executed at this group size. Numbers appearing in the $G=64$ column of the token-budget reanalysis (Appendix C, Table 7) are extrapolated from the inverted-U fit on $G \in \{4, 8, 16, 32\}$ and are descriptive only; we explicitly do not treat $G=64$ as Tier-B evidence.

Total tokens T	Metric	$G=4$	$G=8$	$G=16$	$G=32$	$G=64$
1M	heldout acc.	0.41	0.48	0.47	0.42	0.35
	$\widehat{\text{TBGU}} (\times 10^{-3})$	0.92	1.12	1.05	0.87	0.61
4M	heldout acc.	0.55	0.67	0.69	0.66	0.58
	$\widehat{\text{TBGU}} (\times 10^{-3})$	1.01	1.24	1.31	1.26	0.98
16M	heldout acc.	0.63	0.78	0.82	0.84	0.80
	$\widehat{\text{TBGU}} (\times 10^{-3})$	1.08	1.29	1.36	1.40	1.22
64M	heldout acc.	0.64	0.80	0.85	0.88	0.87
	$\widehat{\text{TBGU}} (\times 10^{-3})$	1.06	1.28	1.38	1.43	1.37

Table 7: Token-budget-normalized G -sweep for GRPO on Qwen3-8B / GSM8K ($K=1$ rollout per sample, $\bar{L} = 384$). **Provenance:** single-seed Wave-2/Wave-6 ablation runs (one Tinker run per $G \in \{2, 4, 8, 16, 32\}$ at `seed=42`, 30 steps; see Appendix A, Group-Size Ablation block); the $G=64$ column is *extrapolated* from the inverted-U fit on $G \leq 32$ and has no direct run. Each cell averages *across token-budget bands* for that G (not across seeds): for each (G, T) we partition the run’s optimizer-visible tokens into bands centred on T and average held-out accuracy / $\widehat{\text{TBGU}}$ within those bands. **Evidence grade:** $B^\dagger$ (single-seed, closed-backend, descriptive); the $G=64$ column is extrapolated and not a Tier-B cell. Bold = best in row. The TBGU-optimal G shifts rightward with larger T : $G=8$ wins at $T=1\text{M}$, $G=32$ wins at $T \geq 16\text{M}$ (the canonical training budget used elsewhere in the paper). The single-seed fixed-step sweep instead places $G=4$ first; we read this as small- G benefiting from more optimizer steps under that very small token budget rather than as a universal optimum. Fit and reanalysis script: `experiments/group_size_token_normalized.py`.

Analysis	r	N	95% CI (bootstrap)	p_{perm}
Pooled original	−0.769	15	[−0.93, −0.52]	0.0004
Pooled extended	−0.443	29	[−0.69, +0.03]	0.0146
Within GSM8K	+0.401	19	[−0.08, +0.83]	0.0837
GSM8K model-varying [†]	+0.126	7	[−0.97, +1.00]	0.8139

Table 8: ZVF–last-10 reward correlations with bootstrap CIs and permutation p -values. The extended pool reports every cell whose step-level ZVF trace is in the released artifact. [†] The $N=7$ nonparametric CI is uninformative; the parametric Fisher- z CI is [−0.69, +0.80], also crossing zero.

Specification	r_{partial}	df	p	Interpretation
Raw pooled, no controls	−0.443	27	0.016	Task is a lurking variable.
+ task FE	−0.026	26	0.896	Task identity removes the sign.
GSM8K within-task	+0.401	17	0.088	Sign flips, CI crosses zero.
GSM8K model-varying	+0.126	5	0.788	Underpowered ($N=7$).
Pooled + task FE + $\log G$	+0.021	25	0.916	Group size does not rescue it.
Pooled + task FE + base acc	+0.135	25	0.502	Pre-RL ability does not rescue it.
Pooled + task FE + $\log G$ + base acc	+0.136	24	0.507	Joint controls; not significant.
GSM8K + $\log G$ + base acc	+0.420	15	0.094	Within-task partial agrees with raw.

Table 9: Partial Pearson correlations of mean ZVF with last-10 reward (`pingouin.partial_corr`). Last-10 reward is the outcome and is never on the right-hand side. Stack fixed effects are not reported (too little within-stack variation: ≤ 3 stacks with ≥ 2 runs).